From a clean cluster to DPU services

NICo Core 2.2.2 + DPF v26.4.0 | Argo / NVIDIA setup.sh | 18 October 2026

PLANNING EDITION. This records the proposed split installation. The DPF-only Argo entry point and complete hybrid rollout have not been validated. Do not combine the existing all-stack NICo Applications with setup.sh.

The goal

Keep NVIDIA setup.sh as the installer for shared prerequisites and NICo. Use external Argo CD for the DPF layer. NICo then creates its own DPU provisioning and service resources. Check HBN traffic and hardware host restrictions separately.

```
Ready management Kubernetes cluster
       |
       +--> NVIDIA setup.sh: shared prerequisites
       |       storage / cert-manager / Vault / PostgreSQL / ESO
       |
       +--> External Argo CD: DPF layer
       |       service Argo / Kamaji / NFD / maintenance / DPF
       |       DPFOperatorConfig / carbide-dpf-cluster
       |
       +--> NVIDIA setup.sh: NICo Core
               |
               +--> NICo hardware onboarding
                       |
                       DPF provisions DPU OS + software
                       |
                       DPU joins Kubernetes -> HBN services
                       |
                       Verify traffic + host restrictions
```

What "from zero" means here

Start with three Ready, schedulable Kubernetes nodes, working CNI/DNS and reachable registries. Installing Kubernetes, switch configuration and server Ubuntu installation are separate prerequisites. DPF provisions the DPU Arm OS; it does not install Ubuntu on the server.

One owner for each resource

NVIDIA setup.sh owns shared prerequisite releases and NICo Core. External Argo owns only the agreed DPF layer. DPF service Argo deploys DPU workloads. NICo owns its generated BFB, flavor, deployment and inventory resources. Do not apply standalone Europe/Asia provisioning or HBN manifests over NICo-owned resources.

This is a customer integration design, not a claim that NVIDIA supplies or certifies this exact Argo deployment workflow.

Step 1 - Prepare the source and settings

Administration machine | No installation in this step

Use the exact NICo source version being qualified. Do not mix the moving online guide with a different release checkout.

```
git clone https://github.com/dsx-ai-factory/infra-controller.git
cd infra-controller
git checkout 80d131a5cd3280f9cef3131fa0c5647bdf3e79d0
cd helm-prereqs
kubectl config current-context
kubectl get nodes
kubectl get storageclass
```

Required tools include Bash, kubectl, Helm, Helmfile and helm-diff, jq, ssh-keygen, envsubst, Python 3 and PyYAML. Follow the pinned NVIDIA prerequisite documentation for installation and supported versions.

Create a private environment file

```
install -d -m 700 "$HOME/nico-private"
umask 077
cat > "$HOME/nico-private/env-vars" <<'ENV'
export NICO_IMAGE_REGISTRY=harbor.runailabs-ps.com/nico
export NICO_CORE_IMAGE_TAG=v2.2.2
export NICO_STORAGE_CLASS=local-path-persistent
export NICO_MANAGE_DEFAULT_STORAGE_CLASS=false
# Optional private registry access; edit locally, never commit:
# export REGISTRY_PULL_USERNAME=YOUR_ROBOT_USER
# export REGISTRY_PULL_SECRET=YOUR_REGISTRY_SECRET
ENV
vi "$HOME/nico-private/env-vars"
chmod 600 "$HOME/nico-private/env-vars"
source "$HOME/nico-private/env-vars"
```

Change the registry, image tag and storage selection to your published artifacts. Source loads shell variables; it does not configure VIPs, routes, NICo pools or credentials by itself. Keep this file outside Git. Do not use shell tracing when loading secrets.

Edit NVIDIA configuration files before setup

helm-prereqs/values.yaml: site name and PostgreSQL/Vault-related chart settings. values/nico-core.yaml: hostname, site TOML, service VIPs, pools, DNS/NTP/PXE and image overrides. values/metallb-config.yaml: allocated address pools and BGP/L2 advertisement. Review operators/values/ for the chosen prerequisite versions and storage behavior.

This initial sequence covers Core only. REST, Flow and RMS are explicitly excluded below; their images, identity, database and workflow configuration need a separate phase.

Step 2 - Install shared prerequisites

NVIDIA owns this layer before Argo installs DPF

This first pass is needed on a clean cluster because DPF depends on shared components such as cert-manager and storage. Letting Argo install these first and later letting setup.sh install them creates competing owners.

```
# Run inside the pinned checkout: helm-prereqs/
source "$HOME/nico-private/env-vars"
bash setup.sh -y \
  --skip-core --skip-rest --skip-flow --skip-rms --skip-dpf
```

The flags are verified in the pinned source. This command is a proposed clean-cluster step, not a completed live test of the hybrid design.

What this pass does

The script prepares local-path storage, PostgreSQL operator, MetalLB, cert-manager, site PKI, Vault initialization/unseal, External Secrets and NICo prerequisite resources. Inspect its storage behavior before use: choosing a storage class or disabling default-class changes does not mean every local-path installation action is skipped.

```
kubectl -n cert-manager get pods
kubectl -n postgres get pods,pvc
kubectl -n vault get pods,pvc
kubectl -n external-secrets get pods
kubectl -n nico-system get pods
kubectl -n postgres get postgresql
kubectl get clusterissuers
```

Stop / continue gate

Continue when the selected storage works, certificates are ready, Vault is unsealed, PostgreSQL is Running and required database/CA Secrets are present. Preserve the Vault recovery material generated by the NVIDIA workflow. Do not publish it in logs, Git or this PDF.

NVIDIA files remain the source of truth

setup.sh defines execution order. helmfile.yaml defines prerequisite releases. values.yaml, values/nico-core.yaml and operator values define settings. The Argo phase must reuse compatible pins and exclude everything this pass owns.

External Argo installation is separate from DPF service Argo. Their namespaces, installation IDs, watched Application namespaces and shared CRD ownership must be explicitly separated.

Step 3 - Install DPF with external Argo

DPF-only Application: preparation and qualification still required

Install external Argo once

```
helm repo add argo https://argoproj.github.io/argo-helm
helm repo update argo
RAW=https://raw.githubusercontent.com/kirson-git/dsx-os/main
helm install external-argo-cd argo/argo-cd \
  --version 9.4.1 --namespace argocd --create-namespace \
  -f "$RAW/NICo/values/external-argo.yaml" \
  --wait --timeout 10m
kubectl -n argocd get pods
```

Skip this Helm installation if external Argo already exists. Do not create a second external release over it.

The DPF-only Application must own

DPF service Argo, Kamaji, Maintenance Operator, Node Feature Discovery, the DPF operator, DPFOperatorConfig and carbide-dpf-cluster. It must not own storage, cert-manager, Vault, PostgreSQL, ESO or NICo Core.

Required NICo-specific DPF settings

```
# DPFOperatorConfig.spec - required integration intent:
dpuDetector:
  disable: true
provisioningController:
  installInterface:
    installViaRedfish:
      skipDPUNodeDiscovery: true
overrides:
  argoCDNamespace: dpf-operator-system
  kubernetesAPIServerVIP: YOUR_REACHABLE_K8S_API_IP
  kubernetesAPIServerPort: 6443

# DPUCluster name expected by the NICo integration:
# carbide-dpf-cluster
# Configure its reserved VIP, interface and CP-node selector.
```

These excerpts show fields, not a complete resource to apply. Reserve the tenant control-plane VIP outside DHCP and independent automatic allocation. NICo owns discovery; do not enable the standalone detector/inventory workflow alongside it.

Do not run the existing all-stack entry point here

NICo/config.yaml and NICo/deploy.sh currently represent the broader Argo-owned stack. DPF/generated/applications/00-platform.yaml also installs shared prerequisites. Neither is a ready DPF-only entry point for this hybrid plan. Its exact apply command must be added after separating and testing ownership.

```
# Read-only checks after the DPF-only Application is implemented:
kubectl -n argocd get applications
kubectl -n dpf-operator-system get pods
kubectl -n dpf-operator-system get dpfoperatorconfigs,dpuclusters
```

Step 4 - Install NICo with setup.sh

Reuse Argo-owned DPF; configure integration explicitly

The important --skip-dpf distinction

In the pinned source, --skip-dpf skips both the DPF installation phase and the automatic Core DPF-off / credential / DPF-on sequence. It does not mean "reuse existing DPF and do all remaining integration automatically".

First bring up Core with DPF integration disabled

Prepare a full copy of NVIDIA values/nico-core.yaml with your site settings. For this first Core pass leave [dpf].enabled false. Keep required service images, PXE artifacts, Unbound images, VIPs and pools configured; the environment file alone is insufficient.

```
# Run from helm-prereqs/ after Step 3 is qualified.
cp values/nico-core.yaml "$HOME/nico-private/nico-core.yaml"
vi "$HOME/nico-private/nico-core.yaml"
source "$HOME/nico-private/env-vars"
bash setup.sh -y --skip-dpf --skip-rest --skip-flow --skip-rms \
  --core-values "$HOME/nico-private/nico-core.yaml"
kubectl -n nico-system get pods
kubectl -n nico-system rollout status deployment/nico-api \
  --timeout=300s
```

This re-runs NVIDIA-owned prerequisites and installs Core. It must not adopt or reinstall the Argo-owned DPF resources.

Then seed credentials and enable integration

Use the release-matched NVIDIA credential workflow to store the site BMC root and required device/factory credentials. The standard automatic BMC step is skipped in this split. Setting NICO_DPF_BMC_ROOT_PASSWORD alone is therefore not enough here. Validate credentials from the management network before ingestion.

```
# Fields within your full nico-core.yaml, not a standalone file:
nico-api:
  dpf:
    rbacCreate: true
    operatorNamespace: dpf-operator-system
  siteConfig:
    nicoApiSiteConfig: |
      # Retain ALL other required site TOML settings.
      [dpf]
      enabled = true
      docker_image_pull_secret = "nico-pull-secret"
```

Explicitly include DPF RBAC and any service chart/image registry overrides: --skip-dpf skips their automatic injection too. Re-run the same setup command with the completed values, then restart nico-api so it reads the changed DPF TOML. This credential/enablement handoff remains a required hybrid integration test.

```
kubectl -n nico-system rollout restart deployment/nico-api
kubectl -n nico-system rollout status deployment/nico-api \
  --timeout=300s
```

Step 5 - Onboard hardware and install DPU services

NICo owns discovery and provisioning requests

Prepare the provisioning network

Provide a dedicated provisioning subnet, DHCP relay where required, DNS, NTP and PXE reachability. NICo uses Kea DHCP. Do not let the old lab DHCP responder and NICo answer the same clients. BMC IP, DPU Arm OS IP, Kubernetes API VIP and host data IP are different addresses.

Onboard approved hardware

Use the pinned NICo expected-machine workflow with real BMC identities/MACs and matching credentials. Approve only the intended servers for ingestion. Those private inventory values are deliberately absent from this public handout.

Follow the provisioning sequence

```
NICo inventory / approved machine
  -> NICo creates DPF deployment and referenced definitions
  -> DPF identifies the DPU via BMC / Redfish
  -> Selected BFB and DPUFlavor drive Arm OS provisioning
  -> Required maintenance / reboot or power cycle
  -> DPU joins carbide-dpf-cluster
  -> DPF service Argo deploys HBN and other selected services
```

```
kubectl -n dpf-operator-system get bfbs,dpuflavors
kubectl -n dpf-operator-system get dpudeployments,dpus
kubectl -n dpf-operator-system get dpus -w
# Ctrl+C stops watching only.
kubectl -n dpf-operator-system describe dpus
kubectl -n dpf-operator-system get events \
  --sort-by=.metadata.creationTimestamp
```

An Argo Synced result proves the requested Kubernetes resources were applied. It does not prove that a BFB installed, that a DPU rebooted or that host Internet access works. Use the DPU conditions and controller logs to locate the current phase.

HBN and zero trust are separate checks

HBN is a DPU networking service. Zero-trust host restrictions limit the server host's ability to control the DPU. Running HBN does not by itself prove those restrictions. Verify the generated deployment/flavor and hardware state. Do not force-edit NICo-owned resources through a standalone Europe/Asia Application.

Hardware generation boundary

The initial profile is based on the BF3 BFB workflow. BF4 uses release-specific BlueFieldSoftware configuration; do not reuse BF3 bundle/flavor assumptions without checking NICo and DPF compatibility.

Step 6 - Validate HBN and zero trust

Acceptance checks | Expected outcomes, not a live status report

Check HBN in the DPU tenant cluster

Use the DPUCluster's issued kubeconfig from a location that can reach its API. Protect the file as a credential. DPU tenant workloads are not the same as pods in the management Kubernetes cluster.

```
# Set this to the issued DPU-cluster kubeconfig, outside Git.
export DPU_KUBECONFIG="$HOME/nico-private/dpu.kubeconfig"
kubectl --kubeconfig "$DPU_KUBECONFIG" get nodes
kubectl --kubeconfig "$DPU_KUBECONFIG" \
  -n dpf-operator-system get pods -l app.kubernetes.io/name=hbn
# Use the actual pod/container names returned by the cluster.
kubectl --kubeconfig "$DPU_KUBECONFIG" \
  -n dpf-operator-system logs POD_NAME -c CONTAINER_NAME --tail=100
```

Prove traffic uses the HBN data interface

Inside each Ubuntu host, select the actual DPU-facing PF/data interface. Its default route, DNS, tenant gateway, fabric routing and external NAT must already be configured.

```
DATA_IF=REPLACE_WITH_DPU_FACING_INTERFACE
ip -4 addr show dev "$DATA_IF"
ip -4 route show
ping -4 -I "$DATA_IF" -c 3 -W 2 cisco.com
curl --noproxy '*' --interface "$DATA_IF" \
  --connect-timeout 5 --max-time 15 -sS -o /dev/null \
  -w 'HTTP=%{http_code} source=%{local_ip}\n' https://www.cisco.com
```

Expected: successful outbound traffic using the intended tenant source address. For isolation, test both directions with known-live destinations and TCP as well as ICMP. Failed ping alone can mean a broken network. Management and IPv6 paths need their own policy checks.

Query actual host restrictions

On the trusted DPU Arm OS with NVIDIA MFT installed, identify the correct device. Replace the path below with the device reported by mst status; do not copy a different generation's device name.

```
sudo mst status -v
sudo mlxprivhost -d /dev/mst/REPLACE_WITH_DEVICE q --full
```

For the intended restricted-host configuration, verify RESTRICTED and the relevant disable_rshim / disable_tracer / disable_port_owner / disable_counter_rd flags. A desired YAML setting is not hardware proof. Required reset/power-cycle completion must also be checked.

The old Europe 10.0.0.2 / Asia 10.0.0.1 HBN demo is a standalone design. Those networks and isolation policies do not automatically transfer to NICo.

Decisions before execution

Planning gates and NVIDIA references

Agreed direction

1. NVIDIA owns shared prerequisites. 2. Argo installs a NICo-compatible DPF layer. 3. Source private environment variables and run NVIDIA setup.sh for Core. 4. Explicitly complete credentials and DPF integration. 5. Onboard approved machines and verify HBN plus host restrictions.

Still required before calling this runnable end to end

Separate a DPF-only Application from the existing all-stack Argo package; verify there are no shared resource owners; validate Kamaji cold start and the two Argo controllers; qualify the --skip-dpf credential/RBAC/enablement handoff; verify published images and site network settings; execute one clean hardware rollout.

If setup.sh later runs inside a pod

Use a Kubernetes Job with pinned NVIDIA source and tools, private Secret inputs, deliberate RBAC and persistent recovery storage. Argo can launch it, but Helm-installed resources will not automatically become native Argo-managed resources. Do not combine the full installer Job with Applications owning those same releases. Pod execution is a later packaging step, not needed to run the script from the administration machine.

Files you edit

Private env-vars: image registry/tag and private credentials. NVIDIA values.yaml: site/prerequisite settings. NVIDIA values/nico-core.yaml: Core networking, images and DPF integration. NVIDIA values/metallb-config.yaml: service VIP pools/routing. DPF-only Argo settings: management API, DPU control-plane VIP/interface and operator configuration. Expected-machine inventory: hardware identities, kept private.

References

NVIDIA NICo Quick Start Guide - setup workflow. Online documentation may advance beyond the pinned release.

Pinned NVIDIA helm-prereqs source - setup.sh, Helmfile and configuration contracts used here.

NVIDIA DPF 26.4 HBN in Zero Trust - DPF deployment reference; do not mix its standalone resources with NICo ownership.

NVIDIA mlxprivhost reference - read-only host restriction queries.

Existing standalone DPF installation and Europe/Asia demo PDF - separate workflow, not the hybrid installation above.

No cluster changes or hardware reboots were performed while preparing this document. No passwords, private keys or hardware MAC addresses are included.